

내일 시험 전, 이 순서로 확인

3시간 · 과목별 3~4문항 · 각 20점 · 총 100점 · 시험 중 자료 열람 불가

빈칸 목적 확인 → 인자 출처 찾기 → 출력 shape 확인 → 최소 코드 작성

먼저 읽을 곳: 3번 Data, 4번 Vision

번호는 공식 범위 표 기준: 1 LLM / 2 RAG / 3 Data / 4 Vision / 5 On-device. 폴더 번호와 다릅니다.

권장 순서: Data 3쪽 → Vision 3쪽 → On-device 2쪽 → LLM 3쪽 → RAG 2쪽.

[기출 복기] 사용자가 제공한 출제 경험·사진에 근거. **[범위 핵심]** 공식 범위와 원본 코드에 따른 선별. 실제 출제 확정 표시는 아닙니다.

인자를 찾는 4개 위치

- ① 함수 입력 def forward(self, x)의 x, __init__(..., hidden_size)의 hidden_size.
- ② 설정값 cfg["emb_dim"], cfg.lr, sequence_length, batch_size. 숫자 대신 주어진 변수 사용.
- ③ 직전 결과 documents → nodes → index / model(x) → logits → loss.
- ④ 다음 줄의 요구 shape cat(dim=1)에 붙일 토큰은 [B,1]; 정답이 [B]면 예측도 [B].

과목별 첫 회상 4묶음 - 우선순위 제안

Data: window-scaler → Conv1D/LSTM → MSE-RMSE → NGCF-BPR.

Vision: Compose 전체 → Attention-잔차 → 분류 loss-Accuracy → U-Net skip-loU.

LLM: QKV-head → embedding-GPT → CE-생성 → 데이터-LoRA-DPO.

RAG: load-index → retrieve-query → llm_input → MCP 도구 연결.

On-device: scale-zero → layer scale 보정 → pruning mask → 증류 loss.

공통 코드 5줄: 이 순서는 손으로 복원

```
optimizer.zero_grad()
logits = model(inputs)
loss = criterion(logits, targets)
loss.backward()
optimizer.step()
```

인자 출처 model-optimizer-criterion: 생성자나 학습 함수 입력. inputs-targets: DataLoader의 현재 배치. loss: 바로 위 손실 계산 결과.

실수 방지 eval()은 평가 모드 전환, no_grad()는 기울기 기록 중지. 목적이 다르며 평가에는 둘 다 사용합니다.

시간 운영 제안

시험 180분: 전체 확인 10분 + 과목별 약 30분 × 5 + 최종 검토 20분. 문항 배점에 따라 조정합니다.

복습은 한 블록을 본 뒤 가리고 1~3줄을 직접 씁니다. 틀린 축·인자만 표시하고 잠들기 전과 내일 입실 전에 다시 씁니다.

우선순위가 낮은 URL·파일명·출력문·그래프 장식의 암기에는 시간을 쓰지 않습니다.

스스로 확인: 인자는 함수 입력인가, 설정값인가, 앞선 결과인가? 손실의 입력·정답 shape가 맞는가?

1 / 3 · Attention과 GPT 부품 연결

Q/K/V에서 문맥 벡터를 만들고, cfg로 GPT의 입출력 차원을 연결한다.

실행: $x \rightarrow Q/K/V \rightarrow \text{score} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow \text{context}$ | 역추적: $\text{context} \leftarrow \text{weights} + V$

[기출 복기] Multi-head Attention의 핵심

```
b, num_tokens, _ = x.shape
keys = self.W_key(x)
queries = self.W_query(x)
values = self.W_value(x)
# Q, K, V 각각에 같은 head 분할을 적용
keys = keys.view(b, num_tokens, self.num_heads,
                  self.head_dim).transpose(1, 2)
queries = queries.view(b, num_tokens, self.num_heads,
                       self.head_dim).transpose(1, 2)
values = values.view(b, num_tokens, self.num_heads,
                     self.head_dim).transpose(1, 2)
attn_scores = queries @ keys.transpose(2, 3)
mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
attn_scores.masked_fill_(mask_bool, -torch.inf)
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1)
attn_weights = self.dropout(attn_weights)
context_vec = (attn_weights @ values).transpose(1, 2)
context_vec = context_vec.contiguous().view(
    b, num_tokens, self.d_out)
context_vec = self.out_proj(context_vec)
```

인자 출처 x: forward 입력 [B,T,Din]. W_key/query/value-mask-dropout-out_proj: __init__에서 생성. num_heads-d_out: 생성자 입력, head_dim=d_out//num_heads. b-num_tokens: x.shape.

실수 방지 head 분할 후 [B,H,T,Dh] → score [B,H,T,T]. softmax는 참고할 토큰 축 -1. 단일 head의 K transpose는 (1,2), multi-head는 (2,3).

근거: Chapter_3_Excercise_Attention.ipynb · CausalAttention / MultiHeadAttention.forward

[기출 복기] cfg에서 만드는 GPT 입출력

```
self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
self.drop_emb = nn.Dropout(cfg["drop_rate"])
self.out_head = nn.Linear(
    cfg["emb_dim"], cfg["vocab_size"], bias=False)
```

인자 출처 cfg: GPTModel(cfg)에 전달된 설정 딕셔너리. vocab_size=ID 종류 수, context_length=최대 위치 수, emb_dim=벡터 길이. Attention 생성 시 d_in/d_out=emb_dim, num_heads=n_heads, dropout=drop_rate, qkv_bias=qkv_bias.

실수 방지 입력 ID [B,T]는 long, 임베딩 [B,T,D]는 float. out_head의 출력은 [B,T,V] 점수(logits)이며 정답 ID가 아닙니다.

근거: Chapter_4_Excercise_GPT.ipynb · TransformerBlock.__init__ / GPTModel.__init__

가리고 쓰기: QK score의 transpose 축 / softmax 축 / Embedding의 두 인자 / 출력 head의 두 차원.

2 / 3 · 정규화, forward, 생성·학습

모델 부품을 실행 순서로 잇고 마지막 토큰 생성과 전체 위치 학습을 구분한다.

실행: ID → embedding → blocks → norm → logits | 역추적: 다음 ID ← 마지막 logits ← GPT

[범위 핵심] LayerNorm과 GPT forward

```
mean = x.mean(dim=-1, keepdim=True)
var = x.var(dim=-1, keepdim=True, unbiased=False)
norm_x = (x - mean) / torch.sqrt(var + self.eps)
return self.scale * norm_x + self.shift

# GPTModel.forward(self, in_idx)의 핵심
batch_size, seq_len = in_idx.shape
tok_embeddings = self.tok_emb(in_idx)
pos_embeddings = self.pos_emb(
    torch.arange(seq_len, device=in_idx.device))
x = self.drop_emb(tok_embeddings + pos_embeddings)
x = self.trf_blocks(x)
x = self.final_norm(x)
logits = self.out_head(x)
```

인자 출처 mean-var: 현재 x에서 계산. eps-scale-shift: LayerNorm.__init__. seq_len: 현재 입력 길이, 최대 길이가 아님. trf_blocks: __init__의 nn.Sequential 안에 쌓인 TransformerBlock들.

실수 방지 LayerNorm은 마지막 D축에서 통계 계산, shape 유지. FFN은 Linear(D,4D) → GELU → Linear(4D,D). 블록은 norm→attention→dropout→잔차 더하기, 이어 norm→FFN→dropout→잔차 더하기.

근거: Chapter_4_Excercise_GPT.ipynb · LayerNorm / FeedForward / TransformerBlock / GPTModel

[기출 복기 + 범위 핵심] 생성과 손실

```
# generate_text_simple(model, idx, max_new_tokens, context_size)
idx_cond = idx[:, -context_size:]
with torch.no_grad():
    logits = model(idx_cond)
logits = logits[:, -1, :]
idx_next = torch.argmax(logits, dim=-1, keepdim=True)
idx = torch.cat((idx, idx_next), dim=1)

# 학습에서는 모든 위치의 정답과 비교
logits = model(input_batch)
loss = torch.nn.functional.cross_entropy(
    logits.flatten(0, 1), target_batch.flatten())
```

인자 출처 idx-context_size: 생성 함수 입력. logits: 모델 반환 [B,T,V]. input_batch-target_batch: loader 배치 [B,T]. V는 logits의 마지막 축에서 유지.

실수 방지 생성: [B,T,V]→[B,V]→[B,1]. 학습 CE: [B*T,V]와 [B*T] long. CE 앞에 softmax를 별도로 넣지 않습니다. keepdim=True는 cat에 필요한 배치·토큰 축 유지.

근거: Chapter_4_Excercise_GPT.ipynb · generate_text_simple / Chapter_5_Excercise_Pretraining.ipynb · calc_loss_batch

가리고 쓰기: (x-평균)/sqrt(분산+eps), 마지막 위치 선택, [B,1] argmax, CE의 두 flatten.

3 / 3 · Dataset, 분류, LoRA·DPO

학습 목적을 보고 입력·정답·학습 파라미터를 구성한다.

실행: 문장 → ID → window/padding → batch → loss | 역추적: loss ← logits + 목적에 맞춘 target

[범위 핵심] 다음 토큰과 분류의 정답 구성

```
# GPTDatasetV1: 함수/생성자 입력 token_ids, max_length, stride
input_chunk = token_ids[i:i + max_length]
target_chunk = token_ids[i + 1:i + max_length + 1]
# Instruction collate: 배치의 최대 길이로 padding한 padded
inputs = torch.tensor(padded[:-1])
targets = torch.tensor(padded[1:])
# 분류 데이터: 먼저 최대 길이로 자르고 남은 길이만 padding
encoded_text = encoded_text[:self.max_length]
encoded_text = encoded_text + [pad_token_id] * (
    self.max_length - len(encoded_text))
```

인자 출처 i: window 반복문. max_length·stride: Dataset 생성자. pad_token_id: collate/데이터셋 함수 입력. self.max_length: 설정값 또는 학습 데이터 최대 길이. labels: 데이터의 Label 열.

실수 방지 next-token target도 [B,T]입니다. 분류 label은 [B]. 분류 head는 Linear(emb_dim,num_classes), loss에는 logits[:, -1, :] 사용. 반복 padding target은 ignore_index로 제외하되 EOS 학습 여부를 원본대로 유지.

근거: Chapter_2_Exercise_Dataset / Chapter_6_Exercise_Finetuning_Classification / Chapter_7_Exercise_Follow_Instructions

[기출 복기] LoRA 행렬과 DPO 선호 방향

```
# LoRALayer.__init__(in_dim, out_dim, rank, alpha)
self.A = nn.Parameter(torch.empty(in_dim, rank))
self.B = nn.Parameter(torch.zeros(rank, out_dim))
# LoRALayer.forward: 기존 Linear에 더할 보정값
delta = (self.alpha / self.rank) * (x @ self.A @ self.B)
# 결합 출력: self.linear(x) + self.lora(x)

# DPO: policy와 reference에서 얻은 응답 log-probability
model_logratios = model_chosen_logprobs - model_rejected_logprobs
reference_logratios = (reference_chosen_logprobs
    - reference_rejected_logprobs)
logits = model_logratios - reference_logratios
losses = -F.logsigmoid(beta * logits)
```

인자 출처 in_dim·out_dim: 원본 Linear의 in_features/out_features. rank·alpha: LoRA 설정. DPO의 네 logprob와 beta: compute_dpo_loss 함수 입력. prompt/chosen/rejected는 같은 데이터 항목에서 가져옵니다.

실수 방지 A는 무작위 초기화, B는 0으로 시작해 초기 보정을 0으로 만듭니다. 기본 weight 고정, LoRA 학습. DPO는 chosen-rejected 방향이며 reference는 고정합니다. LoraConfig가 주어진다면 r·lora_alpha·target_modules를 설정값에서 대응시킵니다.

근거: Chapter_6_Finetuning_for_Text_Classification_LoRA_KR.ipynb · LoRALayer / Chapter_7_Exercise_Follow_Instructions_dpo.ipynb · compute_dpo_loss

가리고 쓰기: window 끝과 target 끝의 차이 / 패딩 개수 / LoRA A·B shape / chosen-rejected 방향.

1 / 2 · 문서에서 검색·답변까지

Reader, Splitter, Index, Retriever, QueryEngine의 입력과 반환값을 연결한다.

실행: documents → nodes/index → retrieve → 답변 | 역추적: 답변 ← 질문 + 검색 문맥

[기출 복기 + 범위 핵심] 로드·청킹·인덱스·실행

```
documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)
# 청킹 규칙을 직접 지정하는 대안
parser = SentenceSplitter(chunk_size=1024, chunk_overlap=200)
nodes = parser.get_nodes_from_documents(documents)
# 또는 from_documents(..., transformations=[text_splitter])
retriever = index.as_retriever()
ret_passages = retriever.retrieve(question)
query_engine = index.as_query_engine()
response = query_engine.query(question)
```

인자 출처 "data": 문제에서 주어진 문서 폴더. documents: Reader 반환 목록. chunk_size·chunk_overlap: 청킹 설정, 숫자는 문제 조건에서 읽기. question: 사용자 질의. index: 앞서 생성한 VectorStoreIndex.

실수 방지 retrieve는 관련 Node 목록, query는 검색과 생성이 합쳐진 응답. nodes는 청킹 결과입니다. chunk 크기 단위는 이 splitter의 토큰 기준이며 overlap < chunk_size로 둡니다.

근거: 1. Llama_index.ipynb · 셀 14~28, 50, QueryEngine 관련 셀

[범위 핵심] CustomQueryEngine 연결

```
# StandardQueryEngine.custom_query(self, query_str)
nodes = self.retriever.retrieve(query_str)
response_obj = self.response_synthesizer.synthesize(query_str, nodes)
return response_obj

# OurCustomQueryEngine.custom_query의 직접 생성 경로
nodes = self.retriever.retrieve(query_str)
context_str = "\n\n".join(n.node.get_content() for n in nodes)
response = self.llm.complete(
    self.qa_prompt.format(context_str=context_str, query_str=query_str))
return str(response)
```

인자 출처 self.retriever: index.as_retriever(). response_synthesizer:

get_response_synthesizer(response_mode="compact"). self.qa_prompt: 앞서 만든 PromptTemplate. format의 키는 템플릿의 {context_str}, {query_str}와 일치.

실수 방지 OurCustomQueryEngine 생성 시 retriever=retriever, response_synthesizer=synthesizer, llm=llm, qa_prompt=simple_qa_prompt. 이 구현은 생성자에 synthesizer를 받아도 custom_query에서는 llm.complete를 직접 씁니다.

근거: 1. Llama_index.ipynb · 셀 76~83, StandardQueryEngine / OurCustomQueryEngine

가리고 쓰기: documents를 받는 함수 / 질문을 받는 함수 / 검색 결과의 text 추출 / format의 두 키.

2 / 2 · llm_input, MCP, 평가

검색 결과를 messages로 조립하고 MCP 도구와 평가 입력을 연결한다.

실행: 검색 chunk → prompt_generator → messages → completion | 역추적: 답변 ← LLM ← references

[기출 복기] Reader의 llm_input

```
# Reader.generate_response(question, query_time, top_k_chunks)
llm_input = self.prompt_generator(question, query_time, top_k_chunks)
completion = oai_client.chat.completions.create(
    model="gpt-3.5-turbo", temperature=0, messages=llm_input
).choices[0].message.content

# prompt_generator(self, query, query_time, top_k_chunks) 안
references = "\n".join(f"- {chunk.strip()}" for chunk in top_k_chunks)
user_message = (f"{references}\nQuestion: {query}"
                f"\nQuery Time: {query_time}")
llm_input = [
    {"role": "system", "content": self.system_prompt},
    {"role": "user", "content": user_message},
]
return llm_input
```

인자 출처 question/query/query_time: 함수 입력. top_k_chunks: retrieval의 검색 결과 문자열 목록. self.system_prompt: Reader.__init__. oai_client: from openai import OpenAI로 만든 client. model 문자열은 문제에서 준 설정 사용.

실수 방지 messages는 문자열 하나가 아닌 role/content 딕셔너리 목록. LlamaIndex OpenAI.complete와 OpenAI SDK chat.completions.create를 섞지 않습니다. 위 prompt 조립은 원본의 핵심만 압축한 예입니다.

근거: 4_RAG_framework_evaluation_with_MCP.ipynb · 셀 40 Reader; Task_1/Task_2의 동일 흐름

[범위 핵심] MCP 도구와 평가

```
mcp_client = BasicMCPClient(external_mcp_server)
mcp_tool = McpToolSpec(client=mcp_client)
tools = await mcp_tool.to_tool_list_async()
# init_agent 안: tools와 llm은 앞에서 준비
self.agent = FunctionAgent(
    name="Agent", tools=tools, llm=self.llm,
    system_prompt=SYSTEM_PROMPT)
self.agent_context = Context(self.agent)

# 평가 함수 입력은 question, ground_truth, prediction
evaluation_result = generate_answer(
    user_prompt=context_template, system_prompt=INSTRUCTIONS)
eval_res = parse_response(evaluation_result)
```

인자 출처 external_mcp_server: 주어진 서버 주소. mcp_tool: 바로 위 client를 감싼 도구 명세.

SYSTEM_PROMPT·INSTRUCTIONS: 앞선 셀 정의. context_template: 질문·정답·모델 답변을 삽입한 문자열.

실수 방지 비동기 결과는 await로 받습니다. 평가 정답 ground_truth를 실제 답변 생성 프롬프트에 넣지 않습니다. API 키·서버 주소 암기보다 인자 연결과 반환값을 우선합니다.

근거: 4_RAG_framework_evaluation_with_MCP.ipynb · 셀 20,43,47

가리고 쓰기: llm_input의 두 role / messages 인자 / .choices[0].message.content / MCP tools 획득.

1 / 3 · 시계열 데이터 준비

과거 T개를 입력으로, 바로 다음 값을 정답으로 만들어 loader에 넣는다.

실행: Open 열 → 시간 분할 → scaling → window → Tensor → loader | 역추적: batch ← X,y ← window

[범위 핵심] 시간 분할과 MinMaxScaler

```
training_data_len = math.ceil(len(df) * train_ratio)
train_data = df[:training_data_len][["Open"]]
test_data = df[training_data_len:][["Open"]]
scaler = MinMaxScaler(feature_range=(0, 1))
train_scaled = scaler.fit_transform(train_data.values)
test_scaled = scaler.transform(test_data.values)
```

인자 출처 df: 다운로드한 시계열 DataFrame. train_ratio: 앞선 셀 설정(원본 0.8). Open: 선택한 특성 열. .values: [날짜 수, 1] 배열. scaler는 바로 위 객체.

실수 방지 train에서만 fit하여 최소-최대 기준을 학습합니다. test는 transform만. test가 train 범위를 넘으면 0~1 밖도 가능합니다. 시계열의 시간 분할과 학습 window 배치의 shuffle=True는 다른 단계입니다.

근거: ts_solution.ipynb · 원본 셀 12,14

[기출 복기] Sliding window → Tensor → DataLoader

```
# sequence_length = 50: 앞선 셀의 설정
def convert_data_into_tensors(data_seq):
    features, labels = [], []
    for i in range(len(data_seq) - sequence_length):
        features.append(data_seq[i:i + sequence_length])
        labels.append(data_seq[i + sequence_length, 0])
    features = torch.tensor(np.array(features), dtype=torch.float32)
    labels = torch.tensor(np.array(labels), dtype=torch.float32)
    return features, labels

def to_loader(x, y, batch_size, shuffle):
    dataset = torch.utils.data.TensorDataset(x, y)
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle)
```

인자 출처 data_seq: train_scaled/test_scaled [N,1]. sequence_length: 함수 밖 설정. i: for 반복 인덱스. label의 0: 단일 특성 Open의 열 인덱스. x,y,batch_size,shuffle: to_loader 입력.

실수 방지 data=[10,20,30,40,50], T=3이면 X=[10,20,30], y=40. 샘플 수=N-T. X [N-T,T,1], y [N-T]; 배치 [B,T,1], [B]. 정답은 마지막 입력 i+T-1이 아니라 그 다음 i+T.

근거: ts_solution.ipynb · 원본 셀 16,18

가리고 쓰기: fit_transform/transform 구분, features 슬라이스, labels 인덱스, dtype, TensorDataset 인자.

2 / 3 · RNN/LSTM·Conv1D와 학습

모델의 출력 전체에서 마지막 시점 하나를 골라 회귀 손실과 연결한다.

실행: [B,T,F] → 모델 → [B,T,1] → [B] 예측 → MSE | 역추적: loss ← 예측 [B] + y [B]

[기출 복기 + 범위 핵심] 모델 생성과 forward

```
# __init__(input_size, hidden_size, num_layers)
self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                    batch_first=True)
self.linear = nn.Linear(hidden_size, 1)
# LSTMModel.forward(self, x)
out, _ = self.lstm(x)
return self.linear(out)

# Conv1DModel.__init__(input_size, hidden_size)
self.conv1d = nn.Conv1d(input_size, hidden_size, kernel_size=2, stride=1)
self.fc = nn.Linear(hidden_size, 1)
# Conv1DModel.forward(self, x)
x = x.transpose(1, 2)
x = self.conv1d(x)
x = x.transpose(1, 2)
return self.fc(x)
```

인자 출처 input_size=X_train.shape[-1] (Open 1개). hidden_size·num_layers: 설정값(64,2). x: loader 입력 [B,T,F]. fc/linear의 입력 차원은 직전 hidden 채널 수.

실수 방지 LSTM: [B,T,F]→[B,T,H]→[B,T,1]. RNN/GRU도 batch_first 인자와 출력 연결은 유사. Conv1D: [B,T,F]→[B,F,T]→[B,H,T-1]→[B,T-1,H]→[B,T-1,1]. 이 원본 forward에는 flatten이 없습니다.

근거: ts_solution.ipynb · 원본 셀 22,30,35

[범위 핵심] 마지막 시점·학습·평가

```
loss_fn = nn.MSELoss(reduction="mean")
optimizer = optim.Adam(model.parameters(), lr=1e-3)
pred = model(batch_x[:, -1, 0])
loss = loss_fn(pred, batch_y)
optimizer.zero_grad()
loss.backward()
optimizer.step()

model.eval()
with torch.no_grad():
    test_predictions = model(X_test[:, -1, 0])
test_predictions = test_predictions.cpu().numpy()
y_test = y_test.cpu().numpy()
rmse = root_mean_squared_error(y_test, test_predictions)
```

인자 출처 model: 생성한 모델. batch_x, batch_y: train_loader의 배치. X_test: 테스트 Tensor. metric 호출의 y_test는 CPU numpy로 바꾼 정답. lr은 문제/설정값.

실수 방지[:, -1, 0]은 모든 배치·마지막 시간·출력 특성 0번 → [B]. 예측 [B,1]과 y [B]를 그대로 MSE에 넣으면 잘못 broadcast될 수 있습니다. RMSE=sqrt(MSE), MAE=평균 절댓값 오차. 원래 가격 단위 평가라면 양쪽 모두 역스케일.

근거: ts_solution.ipynb · 원본 셀 24,28; metric 설명은 공식 범위 보장

가리고 쓰기: nn.LSTM 세 인자의 출처, Conv1D의 두 transpose, pred 슬라이싱, backward→step.

3 / 3 · 추천: NGCF, NCF, 순위 평가

ID를 벡터로 바꾸고, 관측한 아이템의 점수가 더 커지도록 학습한다.

실행: ID/edge → embedding → message → score → BPR | 역추적: 추천 순위 ← score ← user/item 벡터

[범위 핵심] Graph의 ID와 메시지

```
# create_edge_index: user 뒤에 item 노드 배치
dst.append(row["movieid"] + num_users)
# NGCFLayer.forward: edge_index에서 두 끝점 추출
src, dst = edge_index
norm = 1.0 / torch.sqrt(deg[src] * deg[dst])
src_feat = node_features[src]
dst_feat = node_features[dst]
edge_messages_for_dst = self.W1(src_feat) + self.W2(src_feat * dst_feat)
edge_messages_for_dst *= norm.unsqueeze(1)
# NGCF.forward: 각 layer의 표현을 특징 축으로 연결
final_features = torch.concat(layer_outputs, dim=-1)
user_features = final_features[:self.num_users]
item_features = final_features[self.num_users:]
```

인자 출처 row: ratings의 한 행. num_users: 사용자 수. edge_index: [2,E] long. deg: 앞선 index_add_로 집계한 노드 연결 수. W1/W2: 생성자 Linear. layer_outputs: 초기+각 graph layer 결과 목록.

실수 방지 norm [E]를 [E,1]로 바꿔 feature [E,D]에 곱합니다. index_add는 반환값을 받아야 하고 index_add_는 원본 변경. solution의 한 줄에 index_add 반환 누락이 있어 그 오기는 암기하지 않습니다.

근거: RecSys_GCF_sol.ipynb · 셀 10,14,15

[범위 핵심] BPR-NCF-Top-K

```
# bpr_loss(user_emb, pos_item_emb, neg_item_emb, ...)
pos_scores = torch.sum(user_emb * pos_item_emb, dim=1)
neg_scores = torch.sum(user_emb * neg_item_emb, dim=1)
loss = -torch.mean(F.logsigmoid(pos_scores - neg_scores))
# NCF.forward: 두 ID를 별도 Embedding에 넣어 [B,32]씩 조회
input_embedding = torch.cat([user_embedding, movie_embedding], dim=1)
# [B,64] → fc1 → ReLU → fc2 → 평점 [B,1]
topk_scores, topk_indices = torch.topk(scores, k=k)
ndcg_u = dcg / idcg if idcg > 0 else 0.0
```

인자 출처 세 emb: 함수 입력, 모두 [B,D]. scores: 한 사용자의 전체 아이템 점수. k: 평가 함수 입력. dcg/idcg: 앞선 순위 누적값. NCF의 users/movies는 long ID; n_users/n_movies는 생성자 입력.

실수 방지 BPR은 positive-negative의 순서. NCF 원본은 평점 회귀(MSE 계열); 클릭/비클릭 0/1 logit 문제일 때 BCEWithLogitsLoss. Hit@K=정답 하나라도 상위 K에 있다; NDCG는 높은 순위에 더 큰 보상. 원본 NGCF는 Recall-Precision-NDCG 평가.

근거: RecSys_GCF_sol.ipynb · 셀 15,17 / RecSys_NCF.ipynb · Neural_Collaborative_Filtering

가리고 쓰기: item ID offset / norm.unsqueeze(1) / feature concat 축 / BPR 차이 방향 / topk 인자.

1 / 3 · CIFAR 전처리와 ResNet 학습

이미지와 정답을 모델에 맞게 만들고, 분류 손실·정확도에 연결한다.

실행: PIL → 증강 → Tensor [C,H,W] → batch → logits [B,K] | 역추적: 정확도 ← 예측 ID + label

[기출 복기] 학습 transform 전체를 복원

```
train_tfms = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])
test_tfms = T.Compose([
    T.ToTensor(),
    T.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])
train_set = torchvision.datasets.CIFAR10(
    root=data_root, train=True, download=True, transform=train_tfms)
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
```

인자 출처 32: CIFAR 이미지 크기, padding=4: 주어진 증강 설정. CIFAR10_MEAN/STD: 셀 앞 RGB 평균·표준편차. data_root:batch_size: 앞선 설정. transform에는 생성한 변환 객체를 연결.

실수 방지 8-bit PIL → ToTensor는 float [3,32,32], 0~1. Normalize는 채널별 (x-mean)/std이며 출력 범위가 정확히 -2.5~2.5로 고정되지는 않습니다. 테스트에는 임의 crop/flip을 넣지 않습니다.

근거: 01_ResNet18_CIFAR10.ipynb · 셀 6

[범위 핵심] CIFAR용 head·손실·정확도

```
model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
                        bias=False)
model.maxpool = nn.Identity()
model.fc = nn.Linear(model.fc.in_features, num_classes)
criterion = nn.CrossEntropyLoss(label_smoothing=cfg.label_smoothing)
logits = model(images)
loss = criterion(logits, labels)
preds = logits.argmax(dim=1)
acc = (preds == labels).float().mean().item()
```

인자 출처 num_classes=len(train_set.classes), CIFAR는 10. fc.in_features: 기존 모델 head가 받는 특징 수. images,labels: loader의 현재 배치. cfg: TrainConfig 인스턴스.

실수 방지 logits [B,K] float / labels [B] long. CE 전에 softmax 불필요. ResNet 잔차는 $F(x)+\text{shortcut}(x)$, 덧셈 shape가 같아야 합니다. 일반 학습은 zero_grad→backward→step; 원본 AMP는 scaler.scale(loss).backward()→scaler.step(optimizer)→scaler.update().

근거: 01_ResNet18_CIFAR10.ipynb · 셀 9,11,13

가리고 쓰기: Compose 4개 순서 / Normalize 인자 출처 / fc 입력 차원 / argmax 축 / label dtype.

2 / 3 · ViT와 Attention·Encoder

이미지를 패치 토큰으로 바꾸고 head별 Attention 뒤 잔차 연결을 구현한다.

실행: 이미지 → patch → CLS+position → Encoder → class | 역추적: class ← CLS ← attention tokens

[범위 핵심] Patch/CLS/Position

```
num_patches = (image_height // patch_height) * (image_width // patch_width)
patch_dim = cfg.channels * patch_height * patch_width
# 펼친 patch 뒤 nn.Linear(patch_dim, cfg.dim)
x = self.to_patch_embedding(img)
b, n, _ = x.shape
cls_tokens = repeat(self.cls_token, "1 1 d -> b 1 d", b=b)
x = torch.cat((cls_tokens, x), dim=1)
x = x + self.pos_embedding[:, :(n + 1)]
x = self.dropout(x)
x = self.transformer(x)
x = x[:, 0] if self.pool == "cls" else x.mean(dim=1)
return self.mlp_head(x)
```

인자 출처 image/patch 크기·channels·dim: ViTConfig. b,n: patch embedding 결과 [B,N,D]. cls_token: __init__의 [1,1,D] 파라미터. pos_embedding: [1,N+1,D].

실수 방지 CLS는 토큰 축 dim=1에 추가, 위치 개수 N+1. pool="cls"는 0번 토큰 선택 [B,D]. 32×32, patch=4×4, C=3이면 N=64, patch_dim=48 (계산 예시).

근거: 02_ViT_CIFAR10.ipynb · 셀 11 ViT

[기출 복기 관련 핵심] QKV 분리와 Encoder 잔차

```
# Attention.forward: h=self.heads, scale=dim_head**-0.5
qkv = self.to_qkv(x).chunk(3, dim=-1)
q, k, v = map(lambda t: rearrange(t,
    "b n (h d) -> b h n d", h=h), qkv)
dots = einsum("b h i d, b h j d -> b h i j", q, k) * self.scale
attn = self.attend(dots)
out = einsum("b h i j, b h j d -> b h i d", attn, v)
out = rearrange(out, "b h n d -> b n (h d)")
return self.to_out(out)
# Transformer.forward (attn/ff는 PreNorm으로 감싼 부품)
for attn, ff in self.layers:
    x = attn(x) + x
    x = ff(x) + x
```

인자 출처 x: [B,N,D]. self.to_qkv: Linear(dim,3*heads*dim_head). h=self.heads는 생성자 설정. self.attend=Softmax(dim=-1). attn,ff: self.layers 안 부품.

실수 방지 Vision Encoder는 보통 전체 패치를 보므로 GPT의 미래 causal mask를 자동으로 넣지 않습니다. 사진 복기의 DETR EncoderLayer는 현재 03_DETR 노트북에 직접 정의돼 있지 않아, 실제 확인한 ViT Encoder로 공통 구조를 대비합니다. norm 위치는 제시 코드 기준.

근거: 02_ViT_CIFAR10.ipynb · 셀 9 Attention / Transformer; DETR MHA·Encoder는 사용자 기출 사진 근거

가리고 쓰기: CLS concat 축 / N+1 이유 / QKV chunk 축 / head 분리·결합 / residual 두 줄.

3 / 3 · U-Net·DETR·평가 구분

분류·분할·검출의 출력과 필요한 평가식을 구분한다.

실행: image → spatial features → mask 또는 boxes | 역추적: mask ← decoder + encoder skip

[범위 핵심] U-Net Up.forward의 skip

```
# forward(self, x1, x2): x1=decoder, x2=encoder skip
x1 = self.up(x1)
diff_y = x2.size(2) - x1.size(2)
diff_x = x2.size(3) - x1.size(3)
x1 = F.pad(x1, [diff_x // 2, diff_x - diff_x // 2,
               diff_y // 2, diff_y - diff_y // 2])
x = torch.cat([x2, x1], dim=1)
return self.conv(x)
# binary mask: logits와 mask 모두 [B,1,H,W]
criterion = nn.BCEWithLogitsLoss()
```

인자 출처 x1:x2: Up.forward의 두 입력. up-conv: Up.__init__. diff_x/y: 두 Tensor 공간 크기 차이. BCE 정답 mask는 float 0/1, logits와 같은 shape.

실수 방지 U-Net은 채널 concat이므로 dim=1, 채널 수는 합계. ResNet의 add와 다릅니다. 이진 mask는 sigmoid(logits)>threshold로 판정. IoU=교집합/합집합, Dice=2×교집합/(예측 수+정답 수). 0분모는 eps로 처리.

근거: 04_Unet.ipynb · 셀 7 Up, 셀 9 loss; IoU/Dice 식은 공식 평가 범위 보강

[범위 핵심] DETR class-box 후처리

```
img = transform(im).unsqueeze(0)
outputs = model(img)
probas = outputs["pred_logits"].softmax(-1)[0, :, :-1]
keep = probas.max(-1).values > 0.9
bboxes_scaled = rescale_bboxes(outputs["pred_boxes"][0, keep], im.size)
# box_cxcywh_to_xyxy(x), x:[N,4]
x_c, y_c, w, h = x.unbind(1)
b = [x_c - 0.5*w, y_c - 0.5*h,
     x_c + 0.5*w, y_c + 0.5*h]
return torch.stack(b, dim=1)
```

인자 출처 im: 원본 PIL 이미지, im.size=(W,H). outputs: 모델 dict. pred_logits [B,Q,K+1], pred_boxes [B,Q,4]. 0.9: 코드에 주어진 confidence 기준. rescale은 xyxy에 [W,H,W,H]를 곱함.

실수 방지 :-1은 no-object 클래스 제외. 같은 keep mask를 boxes에도 적용. DDPM·Stable Diffusion은 이번 마지막 복습에서 후순위: noise-timestep을 모델에 주고 noise 예측 손실로 연결하는 정도를 먼저 기억합니다.

근거: 03_DETR.ipynb · 셀 8,18

가리고 쓰기: Up의 dim=1 / BCE 입력·정답 shape / IoU와 Dice 분모 / DETR no-object 제외 위치.

1 / 2 · 양자화 / 역양자화

실수↔정수 대응식과 layer 출력의 scale 보정을 복원한다.

실행: float → /scale → round → +zero → clamp | 역추적: float 근사 ← (정수-zero)×scale

[기출 복기 관련 + 범위 핵심] 기본 양자화

```
scale = (fp_max - fp_min) / (quantized_max - quantized_min)
zero_point = round(quantized_min - fp_min / scale)
scaled_tensor = fp_tensor / scale
rounded_tensor = torch.round(scaled_tensor)
rounded_tensor = rounded_tensor.to(dtype)
shifted_tensor = rounded_tensor + zero_point
# 범위를 벗어난 정수는 quantized_min/max로 clamp
# 역양자화 일반식: 먼저 실수형으로 계산
fp_reconstructed = (quantized_tensor.float() - zero_point) * scale
```

인자 출처 fp_tensor-bitwidth-scale-zero_point-dtype: linear_quantize 입력. fp_min/max: fp_tensor.min/max.
quantized_min/max: bitwidth와 signed 여부로 계산. scale와 zero_point 계산 함수의 앞부분을 확인.

실수 방지 signed b-bit 범위 $[-2^{b-1}, 2^{b-1}-1]$. round는 정수 값으로 반올림하되 dtype 변환은 별도. torch.round의 정확한 .5는 가까운 짝수. 10~20을 1000~2000에 대응시키면 scale=.01, zero_point=0.

근거: 2. Quantization for CNN_answer.ipynb · get_quantization_scale_and_zero_point / linear_quantize; 역양자화는 공식식

[기출 복기] quantized_linear / quantized_conv2d

```
# weight scale을 output 채널에 맞게 broadcast
weight_scale = weight_scale.flatten().view(1, -1) # Linear
# Conv2d일 때는 .flatten().view(1, -1, 1, 1)
# 정수 누산 또는 이를 흉내 낸 연산 후
output = output.float()
output = output * (input_scale * weight_scale / output_scale)
output = output + output_zero_point
output = output.round().clamp(
    *get_quantized_range(feature_bitwidth)).to(torch.int8)
# bias를 미리 양자화할 때: bias_scale = input_scale * weight_scale
```

인자 출처 input_scale-weight_scale-output_scale-output_zero_point: layer 함수 인자, 앞서 입력/가중치/출력을 양자화하며 정한 값. output: F.linear/F.conv2d 누산 결과. reshape 기준: 출력 채널 축.

실수 방지 input×weight를 곱했으므로 실제 단위는 두 scale의 곱. output_scale로 나눠 출력 정수 단위로 맞춥니다.
zero-point 보정이 bias에 흡수됐는지 앞 코드를 확인하여 두 번 빼지 않습니다. K-means의 대표값 수는 $2^{bitwidth}$.

근거: 2. Quantization for CNN_answer.ipynb · quantized_linear / quantized_conv2d / bias quantize / k_means_quantize

가리고 쓰기: scale의 분자/분모 / zero_point 부호 / round / input_scale*weight_scale/output_scale.

2 / 2 · Pruning과 Distillation

작은 중요도 가중치를 제거하고 teacher-정답을 student 손실에 연결한다.

Pruning: 중요도 → threshold → mask | Distillation: teacher/student → soft loss + label loss

[범위 핵심] Magnitude pruning

```
num_pruned_elements = round(weight.numel() * sparsity)
importance = torch.abs(weight)
threshold = torch.kthvalue(importance.flatten(), num_pruned_elements)[0]
mask = importance > threshold
weight.mul_(mask)
# LLM의 activation-aware 중요도 (별도 방법)
importance = torch.abs(W) * input_feat[n]
```

인자 출처 weight-sparsity: pruning 함수 입력. numel은 전체 원소 수. threshold: k번째 작은 중요도. W: 현재 Linear weight, input_feat[n]: calibration에서 수집한 현재 layer 입력 특성 통계.

실수 방지 mask=True가 남길 가중치. k=0 또는 sparsity=1은 앞의 경계 처리부터 확인. threshold 동물이면 정확히 k개보다 많이 제거될 수 있음. 0을 만드는 것만으로 실제 연산 시간이 반드시 줄지는 않습니다.

근거: 1. Pruning for CNN_answer.ipynb · fine-grained pruning / 4. Pruning for LLM_answer.ipynb

[범위 핵심] teacher + 정답 → student 학습

```
teacher.eval()
student.train()
with torch.no_grad():
    teacher_logits = teacher(inputs)
    student_logits = student(inputs)
    soft_targets = nn.functional.softmax(teacher_logits / T, dim=-1)
    student_prob = nn.functional.softmax(student_logits / T, dim=-1)
    soft_targets_loss = torch.sum(
        soft_targets * (soft_targets.log() - student_prob.log())
    ) / student_prob.size(0) * (T**2)
    label_loss = ce_loss(student_logits, labels)
    loss = (soft_target_loss_weight * soft_targets_loss
            + ce_loss_weight * label_loss)
    loss.backward()
    optimizer.step()
```

인자 출처 teacher-student-T-두 loss weight: train_knowledge_distillation 입력. inputs,labels: loader 배치. ce_loss=CrossEntropyLoss. optimizer는 student.parameters()로 생성, backward 전에 zero_grad.

실수 방지 soft 분포는 teacher와 student 둘 다 T로 나눔, 마지막 클래스 축 softmax. label_loss에는 student logits와 진짜 labels. 위는 원본 직접 KL 식; KLDivLoss를 쓰는 변형은 첫 입력에 log_softmax, target에 teacher softmax, T**2 보정.

근거: 3. Knowledge Distillation_answer.ipynb · 셀 27 train_knowledge_distillation

가리고 쓰기: abs→kthvalue→mask / teacher만 no_grad / T가 들어가는 세 위치 / student optimizer.